PES UNIVERSITY

100 feet Ring Road, BSK 3rd Stage, Bengaluru 560085

Department of Computer Science and Engineering

(AI & ML)

B. Tech. CSE - 5th Semester

Aug – Dec 2026

UE23CS351A

DATABASE MANAGEMENT SYSTEMS (DBMS)

PROJECT REPORT on

ScholarHub – Student Scholarship Management & Eligibility System

Submitted by:

SRN	Name	Section
PES1UG24AM096	Diya Agarwal	B
PES1UG24AM098	G Ganikhasri	B

Class of Prof. Smitha GR

[Lab Faculty]

Abstract

ScholarHub is a Student Scholarship Management and Eligibility System designed to centralize scholarship discovery, eligibility checking, applications, document verification, awards, notifications, and administrative reporting. The project uses PostgreSQL as the core relational database for transactional and structured data and MongoDB for flexible, schema-light information such as scholarship extras, reviews, activity logs, and notifications.

The main feature is a SQL-based eligibility engine that evaluates a student's CGPA, annual family income, category, gender, state, course, year of study, disability status, and minority status against scholarship-specific criteria. The system also provides gap analysis for scholarships where exactly one criterion is unmet, a priority score for ranking eligible scholarships, and a consolidated document checklist.

The backend is implemented using Node.js and Express, with PostgreSQL accessed through the pg package and MongoDB accessed through Mongoose. A vanilla JavaScript single-page frontend communicates with REST API endpoints. Database concepts demonstrated include primary and foreign keys, CHECK and UNIQUE constraints, joins, nested and correlated subqueries, aggregation, HAVING and FILTER, window functions, views, SQL functions, stored procedures, triggers, and indexes.

Table of Contents

Sl. No	Topic	Page No.
1	Introduction – Problem Statement	4
2	User Requirements Specification – Tech Stack	5
3	E-R Model	6
4	Relational Model	7
5	SQL DDL Statements	8
6	SQL DML Statements	9
7	NoSQL Database Commands	10
8	Results	11
9	Project Tree Structure	12
10	Sample Application Code and Demo	13
11	Evaluation Rubrics – Completion Status	14

1. Introduction

Problem Statement

Scholarship information is often distributed across multiple sources, while eligibility conditions depend on academic, financial, demographic, geographic, and course-specific criteria. Students may therefore spend significant time searching for suitable scholarships and determining whether they qualify. Administrators and verification officers also need a structured way to manage applications, documents, status changes, awards, and reports.

Short Description

ScholarHub addresses this problem by maintaining a centralized scholarship database and matching each student's profile against eligibility criteria stored in PostgreSQL. The application supports four roles: student, administrator, verification officer, and system administrator. Students can register, maintain their profile, browse scholarships, view eligibility, apply, upload document information, track application status, view notifications, and rate scholarships. Staff roles receive interfaces for scholarship management, application review, student information, awards, reports, and system-level monitoring.

The implementation deliberately keeps the eligibility logic in SQL rather than using AI/ML. This makes the eligibility decision deterministic and auditable because each failed criterion can be returned as an explicit reason.

2. User Requirements Specification

Functional Requirements

• Student registration and login with role-based access.

• Student profile management containing CGPA, income, category, gender, state, course, year, disability status, and minority status.

• Scholarship browsing with filters for course, state, category, gender, income, scholarship type, and deadline status.

• Live eligibility computation using PostgreSQL rules.

• Gap analysis showing scholarships for which exactly one eligibility criterion is failed.

• Application submission with required-document slots.

• Document upload metadata and verification workflow.

• Application status tracking and automatic status history.

• Approval and award creation with payment information.

• Scholarship reviews, notifications, activity logging, and reporting.

Tech Stack – Justification

Technology	Purpose	Justification
PostgreSQL	Relational database	Strong constraints, joins, transactions, procedures, triggers and reporting are required for structured scholarship/application data.
MongoDB	NoSQL database	Flexible documents are suitable for FAQs, provider-specific extras, reviews, append-only activity logs and notifications.
Node.js + Express	Backend/API	Provides REST endpoints and integrates both PostgreSQL and MongoDB in one server.
pg	PostgreSQL driver	Direct access to SQL queries, views, functions and procedures.
Mongoose	MongoDB ODM	Defines flexible document models and simplifies MongoDB access.
Vanilla JavaScript + HTML/CSS	Frontend	Lightweight single-page interface served directly by the backend; no framework dependency is required by the actual project.
JWT + bcryptjs	Authentication	JWT carries authenticated session information while bcryptjs handles password hashing/verification.

Choice of NoSQL

MongoDB is used because several parts of the system are naturally schema-light. Scholarship extras can contain provider-specific tags, FAQs, application steps, links and arbitrary extra fields. Reviews contain free-text comments and can grow independently. Activity logs are append-only records whose details

may differ by action, while notifications are per-user documents with a simple flexible structure. PostgreSQL remains the source for core relational keys and transactional relationships.

3. E-R Model

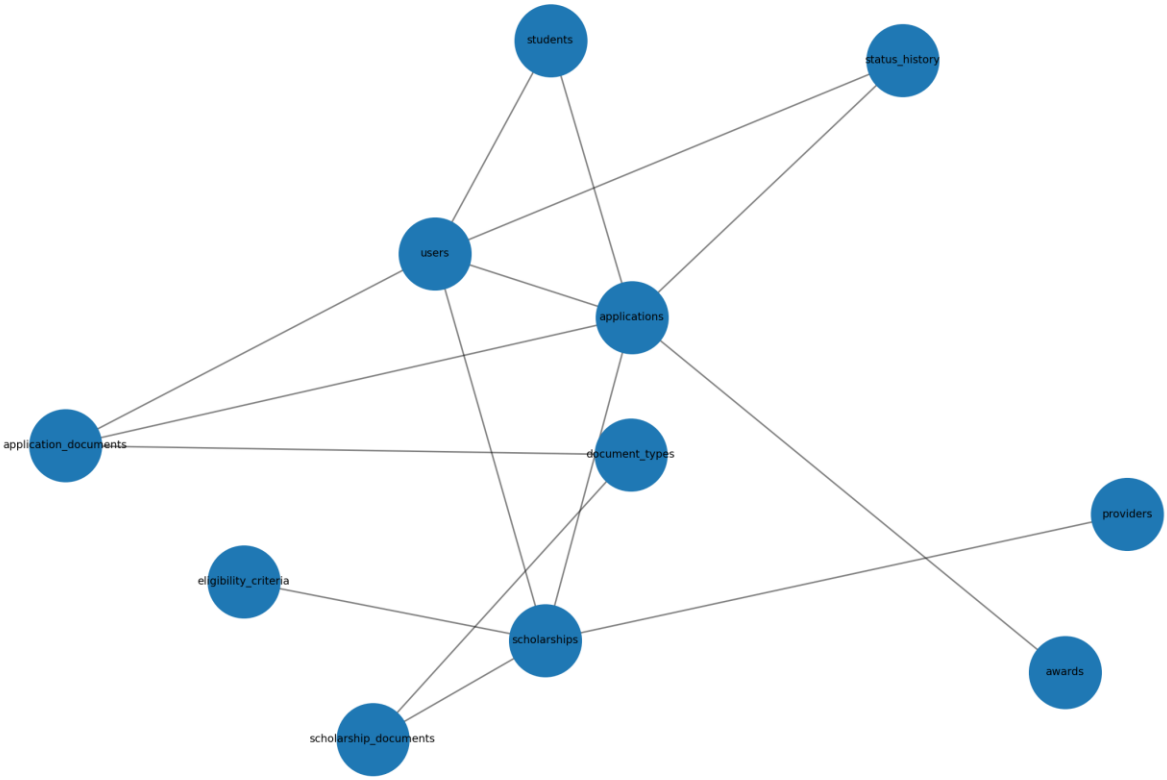
List of Entities

Entity	Type	Purpose
Users	Strong	Authentication accounts and roles
Students	Strong	Student profile; 1:1 with a student user
Providers	Strong	Organizations providing scholarships
Scholarships	Strong	Scholarship offerings and deadlines
Eligibility_Criteria	Strong	1:1 eligibility rule set for a scholarship
Document_Types	Strong	Reusable document definitions
Scholarship_Documents	Associative	M:N relation between scholarships and document types
Applications	Strong	Student applications for scholarships
Application_Documents	Strong	Documents attached to an application
Awards	Strong	Award/payment record for an approved application
Status_History	Strong	Audit trail of application status changes

Non-simple / Composite Attributes

The relational implementation represents multi-valued eligibility properties as PostgreSQL arrays: allowed_categories, allowed_genders, allowed_states, and allowed_courses. These are logically multi-valued attributes. The eligibility criteria also contains a grouped set of conditions including minimum CGPA, maximum income, year range, disability requirement and minority requirement.

ER Diagram Snapshot



ERD from Tool-like Representation

The diagram above is generated directly from the relationships defined in database/postgres/01_schema.sql. The associative entity scholarship_documents resolves the many-to-many relationship between scholarships and document types.

4. Relational Model

List of Relations

Relation	Primary Key	Important Foreign Keys / Constraints
users	user_id	email UNIQUE; role CHECK
students	student_id	user_id UNIQUE → users; CGPA/income/year CHECK
providers	provider_id	name UNIQUE; provider_type CHECK
scholarships	scholarship_id	provider_id → providers; deadline > start
eligibility_criteria	criteria_id	scholarship_id UNIQUE → scholarships
document_types	doc_type_id	doc_name UNIQUE
scholarship_documents	(scholarship_id, doc_type_id)	M:N associative relation
applications	application_id	student_id → students; scholarship_id → scholarships; UNIQUE(student_id, scholarship_id)
application_documents	app_doc_id	application_id → applications; doc_type_id → document_types
awards	award_id	application_id UNIQUE → applications; transaction_ref UNIQUE
status_history	history_id	application_id → applications

Schema Diagram

The relational schema consists of 11 PostgreSQL tables. Primary keys uniquely identify records, foreign keys maintain referential integrity, CHECK constraints validate domain values, UNIQUE constraints prevent duplicates, and indexes support common filters and joins.

```
users(user_id PK, email UQ, password_hash, role, created_at)
students(student_id PK, user_id FK/UQ, full_name, gender, category, state,
         course, year_of_study, cgpa, annual_income, is_disabled, is_minority)
providers(provider_id PK, name UQ, provider_type, website)
scholarships(scholarship_id PK, provider_id FK, name, amount, type,
             application_start, deadline, is_active, created_by FK)
eligibility_criteria(criteria_id PK, scholarship_id FK/UQ, min_cgpa,
             max_income, allowed_categories[], allowed_genders[],
             allowed_states[], allowed_courses[], min_year, max_year,
             disability_required, minority_required)
document_types(doc_type_id PK, doc_name UQ)
scholarship_documents(scholarship_id PK/FK, doc_type_id PK/FK)
applications(application_id PK, student_id FK, scholarship_id FK,
             status, applied_on, reviewed_by FK, remarks)
application_documents(app_doc_id PK, application_id FK, doc_type_id FK,
             file_name, submitted_on, verification_status, verified_by FK, verified_at)
awards(award_id PK, application_id UQ/FK, amount, payment_mode,
       payment_status, payment_date, transaction_ref UQ)
status_history(history_id PK, application_id FK, old_status, new_status,
               changed_by FK, changed_at)
```

5. SQL DDL Statements

The DDL is implemented in database/postgres/01_schema.sql. It creates the PostgreSQL schema and indexes. The project contains 11 relational tables.

Representative DDL

```
CREATE TABLE scholarships (
  scholarship_id SERIAL PRIMARY KEY,
  provider_id INT NOT NULL REFERENCES providers(provider_id),
  name VARCHAR(150) NOT NULL,
  amount NUMERIC(10,2) NOT NULL CHECK (amount > 0),
  scholarship_type VARCHAR(15) NOT NULL
    CHECK (scholarship_type IN ('Merit','Need','Minority','Women','Disability','General')),
  application_start DATE NOT NULL,
  deadline DATE NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  created_by INT REFERENCES users(user_id),
  CONSTRAINT chk_dates CHECK (deadline > application_start)
);

CREATE TABLE applications (
  application_id SERIAL PRIMARY KEY,
  student_id INT NOT NULL REFERENCES students(student_id) ON DELETE CASCADE,
  scholarship_id INT NOT NULL REFERENCES scholarships(scholarship_id),
  status VARCHAR(15) NOT NULL DEFAULT 'Submitted'
    CHECK (status IN ('Submitted','Under Review','Approved','Rejected')),
  applied_on TIMESTAMP NOT NULL DEFAULT now(),
  reviewed_by INT REFERENCES users(user_id),
  remarks TEXT,
  CONSTRAINT uq_student_scholarship UNIQUE (student_id, scholarship_id)
);
```

Database Objects

The schema also creates indexes on student course, scholarship deadline, application student, scholarship, status, and application-document relationships. Additional DDL in 02_views_functions.sql, 03_procedures_triggers.sql and the seed script creates views, functions, procedures and triggers.

6. SQL DML Statements

The project uses INSERT, UPDATE and SELECT operations throughout the seed data, backend routes and database procedures. The demo query file database/postgres/05_queries.sql demonstrates joins, nested and correlated subqueries, aggregates, HAVING, window functions, views, functions and trigger/procedure demonstrations.

Sample DML / Query Statements

```
-- Eligible scholarships for student 1
SELECT scholarship_name, amount, deadline_status, days_left
FROM fn_eligibility_check(1)
WHERE is_eligible;

-- Nested query
SELECT DISTINCT st.full_name
FROM students st
JOIN applications a ON a.student_id = st.student_id
JOIN scholarships s ON s.scholarship_id = a.scholarship_id
WHERE s.amount > (SELECT AVG(amount) FROM scholarships);

-- Correlated subquery
SELECT full_name
FROM students st
WHERE NOT EXISTS (
  SELECT 1 FROM applications a
  WHERE a.student_id = st.student_id
);

-- HAVING
SELECT s.name, COUNT(*) AS apps
FROM applications a JOIN scholarships s USING (scholarship_id)
GROUP BY s.name
HAVING COUNT(*) > 1;

-- Window function
SELECT provider_name, name, amount,
       RANK() OVER (
         PARTITION BY provider_name ORDER BY amount DESC
       ) AS rnk
FROM v_scholarship_status;
```

Functions, Procedures and Triggers

fn_eligibility_check(student_id) evaluates every open scholarship and returns eligibility, failed reasons and application status. sp_apply_scholarship performs eligibility validation, creates the application and creates required-document slots. sp_review_application updates the application and creates an award atomically when the status becomes Approved.

Six trigger mechanisms are implemented: application-window validation, verified-document-before-approval, status-history audit, award guard, document verification validation, and automatic students.updated_at maintenance.

7. NoSQL Database Commands

MongoDB database: scholarship_nosql. The project defines four collections: scholarshipextras, reviews, activitylogs and notifications. PostgreSQL primary keys are stored as scholarshipId/userId/entityId where cross-database linkage is required.

Collection	Purpose
scholarshipextras	Flexible tags, how-to-apply steps, FAQs, links, tips and provider-specific extra fields.
reviews	Student ratings and free-text scholarship comments.
activitylogs	High-volume append-only audit/activity events with varying details.
notifications	Per-user in-app alerts with read/unread state.

Representative MongoDB Commands

```
use scholarship_nosql

db.scholarshipextras.find({ tags: "women" })

db.reviews.aggregate([
  { $group: {
      _id: "$scholarshipId",
      avgRating: { $avg: "$rating" },
      n: { $sum: 1 }
  }}
])

db.activitylogs.aggregate([
  { $group: { _id: "$action", count: { $sum: 1 } } },
  { $sort: { count: -1 } }
])

db.notifications.find({ userId: 5, read: false })
```

No vector database is present in the supplied project. The NoSQL implementation is MongoDB and is used for flexible document-oriented features rather than vector search.

8. Results

The supplied project includes seed data for eight students, seven providers, thirteen scholarships and eight document types. The seed data is designed with dates relative to CURRENT_DATE so that the demonstration contains active, expiring, upcoming and closed scholarship cases.

Resulting Data – Representative Tables

Table	Seeded Records / Structure
users	11 accounts: 3 staff accounts + 8 student accounts
students	8 student profiles
providers	7 providers
scholarships	13 scholarship records
eligibility_criteria	13 criteria records, one per scholarship
document_types	8 document types
applications	Demo applications generated through stored procedures in the seed script
application_documents	Required-document slots generated for applications
awards	Created for approved demo applications
status_history	Automatically populated by the status-history trigger

Example Scholarship Results from Seed Data

Scholarship	Amount	Type	Provider
Post Matric Scholarship for SC/ST	₹12,000	Need	National Scholarship Portal
Central Sector Merit Scholarship	₹20,000	Merit	National Scholarship Portal
Pragati Scholarship for Girls	₹50,000	Women	AICTE
Tata Trusts Medical Grant	₹75,000	Need	Tata Trusts
Reliance Foundation UG Scholarship	₹2,00,000	Merit	Reliance Foundation
PES Merit Excellence Award	₹40,000	Merit	PES University
Disability Empowerment Scholarship	₹25,000	Disability	National Scholarship Portal
Tata Women in Tech Fellowship	₹60,000	Women	Tata Trusts

These are source-derived seed records rather than claims of a live database execution. Actual query-result screenshots can be inserted after running the supplied SQL files in PostgreSQL and the MongoDB seed commands.

9. Project Tree Structure

The supplied application does not use Next.js. The actual frontend is a vanilla JavaScript single-page application served by the Express backend. The project tree is:

```
DBMS-Mini_Project-main/
├── README.md
├── frontend/
│   ├── index.html
│   ├── css/
│   │   └── style.css
│   └── js/
│       └── app.js
├── backend/
│   ├── package.json
│   ├── package-lock.json
│   ├── .env.example
│   ├── scripts/
│   │   ├── initPostgres.js
│   │   └── seedMongo.js
│   └── src/
│       ├── server.js
│       ├── config/
│       │   ├── postgres.js
│       │   └── mongo.js
│       ├── middleware/
│       │   └── auth.js
│       ├── models/
│       │   └── index.js
│       ├── routes/
│       │   ├── auth.js
│       │   ├── scholarships.js
│       │   ├── eligibility.js
│       │   ├── applications.js
│       │   ├── verifier.js
│       │   ├── admin.js
│       │   ├── reports.js
│       │   ├── students.js
│       │   └── notifications.js
│       └── utils/
│           ├── helpers.js
│           └── logger.js
└── database/
    ├── postgres/
    │   ├── 01_schema.sql
    │   ├── 02_views_functions.sql
    │   ├── 03_procedures_triggers.sql
    │   ├── 04_seed.sql
    │   └── 05_queries.sql
    └── mongodb/
        ├── README.md
        └── seed_data.json
```

10. Sample Application Code and Demo

Backend API Integration

```
const app = express();
app.use(cors());
app.use(express.json());

app.use('/api/auth', require('./routes/auth'));
app.use('/api/students', require('./routes/students'));
app.use('/api/scholarships', require('./routes/scholarships'));
app.use('/api/eligibility', require('./routes/eligibility'));
app.use('/api/applications', require('./routes/applications'));
app.use('/api/verifier', require('./routes/verifier'));
app.use('/api/admin', require('./routes/admin'));
app.use('/api/reports', require('./routes/reports'));
app.use('/api/notifications', require('./routes/notifications'));
```

Frontend Eligibility Call

```
const [el, docs] = await Promise.all([
  api('/eligibility/me'),
  api('/eligibility/me/checklist')
]);
```

The frontend displays eligible, almost-eligible and not-eligible scholarships. It also shows priority scores, deadline status, application status, document verification and awards.

Authentication and Roles

The supplied frontend defines four role-specific navigation menus: student, verifier, admin and sysadmin. The backend uses authentication middleware and JWT-based authorization. The demo seed provides accounts for each role.

Demo Setup

```
1. Create PostgreSQL database: scholarship_db
2. Run 01_schema.sql
3. Run 02_views_functions.sql
4. Run 03_procedures_triggers.sql
5. Run 04_seed.sql
6. Start MongoDB
7. cd backend
8. npm install
9. npm run db:seed-mongo
10. npm start
11. Open http://localhost:5000
```

The README specifies the demo password as password123 for the supplied demonstration accounts. In a real deployment, these credentials should be replaced and environment secrets must not be committed.